

Component Attribution in Agentic Knowledge Graph Question Answering: A Controlled Ablation, Budget Analysis, and Failure Census

Md. Sakif Khan^{a,*}, Sadia Sharmin^a

^a*Department of Computer Science and Engineering, Bangladesh University of Engineering and Technology, Dhaka, Bangladesh*

Abstract

Large language models answer factual questions fluently, but they sound just as fluent when they are wrong. To fix this, researchers introduced retrieval-augmented generation, which grounds answers in retrieved text. But it retrieves only once, so a question spanning several facts is answered from that one query. Systems that navigate a knowledge graph do better. They interleave retrieval with reasoning, one hop at a time. Such a system has several parts. A planner breaks the question into steps. A scorer picks the next edge. A backtracker recovers from dead ends. A checker reads the draft answer. Papers report the system as a whole. So when it beats a baseline, nobody knows which part earned the gain. We built one, Agentic Graph Reasoning (AGR), with four removable parts, and took it apart. AGR and four baselines share one frozen backbone and one call budget on 400 questions each from WebQSP and ComplexWebQuestions. AGR leads the agentic baseline by 9.5 and 8.7 Hits@1 points. But the whole margin comes from questions where the shared budget cuts the baseline off. Where it finishes, the baseline is ahead. Removing the planner improves WebQSP F1 by 0.083 and cuts token use by 31%. The other three parts, claim verification among them, show no effect our tests could detect. A hand-read census of the failures finds label defects in 5.5% and 4.8% of the samples and names the *echo attractor*. It is a true entity one hop from the answer, so every grounding check passes it.

Keywords: knowledge graph question answering, large language models, agentic retrieval, ablation study, component attribution, benchmark quality

*Corresponding author

Email address: sakifkhanopu@gmail.com (Md. Sakif Khan)

1. Introduction

Large language models answer factual questions fluently, and often enough they answer them wrongly. A fabricated answer reads in the same register as a correct one, so it is hard to catch from the output alone [1], and the problem compounds once an answer requires several facts put together: if the first hop lands on the wrong entity, nothing later in the chain will know. Retrieval-augmented generation [2] grounds the model in retrieved text, but it retrieves once, before any reasoning starts, and the passage that holds an intermediate entity need not resemble the question at all. A knowledge graph [3] turns composition into traversal, since each hop is an edge, and static graph retrieval [4] still fetches a neighbourhood in one shot. Agentic systems close that gap by interleaving retrieval with reasoning: the model takes one step, looks at what it found, and decides where to look next [5, 6, 7, 8].

These systems are assemblies. A typical one decomposes the question into sub-goals, scores candidate edges with the model, backtracks from dead ends, and checks its own output, and the literature reports the assembly as a whole. When it outperforms a static baseline, three things could have produced the gain: a component, the budget the comparison ran under, or the labels of the benchmark itself. The report rarely separates them, so later work inherits the whole assembly on faith, along with its cost. The exception shows why taking a system apart matters: KBQA-o1’s own ablation removes its tree search, and GrailQA F1 falls by thirty points [8]. This paper does the same for an agentic knowledge-graph question-answering system built so that each of its components can be removed cleanly, and it asks which of the three sources produced the gain.

The system is **Agentic Graph Reasoning** (AGR), an explicit state machine over a constrained graph tool API in which a program runs the loop and the language model is consulted only at bounded decision points. Four components can be removed: a *planner*, an *edge scorer*, a *backtracker*, and a *structural verification layer* that splits each draft answer into atomic claims and checks them against the traversed triples before the answer is emitted. The last is the component the system was designed around, and its two graph routes ask whether an edge joins a claim’s endpoints, not whether it is the edge the claim asserts, so a claim that X is Y’s mother is certified by an edge recording that X is Y’s child. Sections 3.2 and 8.1 bound it further.

On 400 questions from each of WebQSP [9] and ComplexWebQuestions [10], over a Freebase-derived environment of 2.59 million entities and 8.31 million triples, AGR reaches Hits@1 of 0.755 and 0.522 and leads four baselines that share its backbone and its call budget. That is where the analysis starts, not where it ends. Against a reimplementation of Think-on-Graph [5] on the same tools, the entire margin comes from the 29% and 44% of questions on which the shared 25-call cap truncates the baseline; where the cap lets it finish, it is ahead (Section 5.3). The ablation finds one component with a detectable effect, and the effect runs the wrong way: removing the planner improves WebQSP F1 by 0.083 while cutting tokens by 31%, because decomposing a question that has no compositional structure throws away the context that would have answered it. Backtracking, model scoring, and claim verification show no effect at the power available, and Section 6.3 states the smallest effect each test could have caught. The failure census finds label defects on 5.5% and 4.8% of the sampled questions, and names a failure that passes every grounding check and that consensus-based label repair cannot see.

2. Related Work

Three lines of work bear on question answering over knowledge graphs: what retrieval fixed before reasoning can reach, what interleaving retrieval with reasoning buys, and what checking an answer against evidence adds. None establishes which component of an agentic system is responsible for its behaviour, and that gap is the one this paper addresses.

Retrieval-augmented generation [2] embeds the question, retrieves similar passages, and answers from them. Where the answer must be composed across passages the ranking signal measures the wrong thing, because the passage carrying the intermediate entity resembles neither the question nor anything the system yet knows to look for; surveys of the graph–language-model intersection identify this as the central limitation of flat retrieval [11]. Knowledge graphs make the missing structure explicit, so composition becomes a traversal: GraphRAG [4] retrieves structured context in place of passages, and a second family keeps retrieval static but learns it, scoring candidate subgraphs with a graph neural network [12] or retrieving a bounded subgraph with a light-weight scorer [13]. All of them retrieve once, before reasoning, so a fixed-radius neighbourhood is either too small to contain the answer

or so large that the relevant edges are lost among thousands of irrelevant ones.

Interleaving retrieval with reasoning removes that constraint. Think-on-Graph [5] established the pattern with model-guided beam search over relations and entities, and its successor adds document retrieval at each step [14]. Plan-on-Graph [6] added decomposition into sub-objectives and reflection-driven backtracking; AGR’s planner and backtracker are those two mechanisms re-hosted in a program-orchestrated loop, its verification layer is the addition, and Section 8.2 records that Plan-on-Graph itself is not among the systems run here. Reasoning-on-Graphs (RoG) [7] instead generates grounded relation paths as explicit plans with a fine-tuned model and traverses them, so answers are faithful by construction, as does graph-constrained reasoning [15]; RoG is also the system whose published subgraphs this paper’s environment is built from (Section 4.1), and Appendix D.2 compares the two directly. KBQA-o1 [8] is closest to AGR’s search strategy, performing agentic knowledge base question answering with full Monte Carlo Tree Search over a ReAct-style [16] agent. AGR does none of this: its search is guided best-first with a bounded beam and backtracking, carrying neither rollouts nor value backpropagation.

A parallel line checks generated content after the fact. Chain-of-Verification [17] has a model draft a response, plan verification questions, answer those independently and regenerate, checking against the model’s own knowledge rather than an external symbolic source, whose limits are documented [18]. A second group critiques the *retrieval*, emitting reflection tokens that judge whether a passage supports the output [19] or grading retrieved documents and triggering a fallback search [20], while FActScore [21] decomposes generations into atomic facts as a metric rather than a control step. The layer of Section 3.2 sits between these lines: it decomposes a draft into claims and checks them against an external source, but the source is the set of triples the agent itself traversed and the check runs before the answer is emitted. The path-faithful systems [7, 15] make the question moot by never generating a claim the graph did not produce. To our knowledge no graph-navigating agent that generates its answer freely checks that answer’s claims against its own traversal before emission, and whether that check earns its place is what Section 6 measures.

The gap that motivates this paper is therefore methodological. Agentic systems are assemblies, and the literature reports them as wholes, so a published gain over a static

baseline rarely establishes which mechanism produced it, whether some contribute nothing, or how much is the budget regime the comparison ran under. The exception is instructive: KBQA-o1’s own ablation removes MCTS while leaving every other component intact, and GrailQA F1 drops from 78.5 to 48.5 [8] — a thirty-point attribution to a single mechanism, obtained precisely because the authors ablated. Section 6 applies the same instrument to AGR, and Section 5.3 adds the second source of a reported gain, the budget, which an ablation cannot see.

3. The Instrument: AGR

AGR inverts the usual agent pattern. It does not hand the model a set of tools through native function-calling and let the model drive the loop. A program orchestrates every step, calls the graph tools deterministically, and consults the model only at bounded decision points, through prompts constrained to return JSON. Every graph operation is therefore logged by construction rather than reconstructed from the model’s account of what it did, and budgets are enforced by counters a router checks rather than requested in a prompt the model may ignore. The consequence this paper uses is that each of the loop’s four components can be removed by a configuration flag without touching the others, which is what Section 6 does; Appendix B.6 states exactly what each flag removes.

3.1. State Machine

Figure B.1, in Appendix B, gives the topology: six agent nodes over a shared typed state, with three cycles. The **Planner** decomposes the question into an ordered chain of at most four sub-objectives in one model call, from a prompt whose first exemplar is a single-hop question that is *not* decomposed. **Explorer** and **Evaluator** form a search loop: the explorer expands the frontier by scoring every candidate relation of at most five anchors, blending embedding similarity against the active sub-objective with a batched model judgement at a frozen $\alpha = 0.7$, and the evaluator judges whether that objective is satisfied from the thirty most similar traversed triples. It may accept only entities the graph returned. The **Backtracker** restores an earlier, higher-scoring frontier and bans the edges that led away from it; that ban list is what turns backtracking into a search operation, since without it a deterministic scorer facing the same restored frontier would select the same

edges again. Appendix B.1 gives the four graph operations, Appendix B.2 the scoring rule, and Appendix B.3 the router and the backtracker’s snapshot discipline. Every path from Evaluator to Answerer passes through the **Verifier**, and the budgets of Appendix B.5 bound all three cycles, so no sequence of model outputs can produce a non-terminating run.

3.2. Structural Verification Layer

Nothing in the navigation loop constrains what the final answer *asserts*. The verifier is the check that does, and it is the component the system was designed around. One model call drafts prose from the traversed triples, extracts the entities the draft asserts as its answer, and decomposes the draft into atomic claims $\langle h, r, t \rangle$ over entity *names*, since the model has never seen a graph identifier. Each claim is then routed through up to three tests and leaves at the first that settles it: **traversed adjacency**, which asks whether the claim’s endpoints were walked together and is the only route of the three that attaches citable evidence; **verify_connection**, which asks the full graph the same question for endpoints that were reached but never walked together; and, for whatever clears neither, an **entailment check** against the fact window the drafter saw. Symbols decide, and the model is consulted only where the graph is silent. If no claim is unsupported the draft is emitted verbatim; otherwise the verifier either sends the agent back to explore for the missing support or, when the budget is spent, rewrites the draft around a deterministically filtered entity list. Appendix B.4 gives the routing and the three outcomes in full.

What the graph routes actually check bounds every claim made for this layer. Both of them ignore the relation and the direction, so what they establish is that the endpoints of a claim are connected in the environment, not that the claimed relation is the one the edge records: a claim that X is Y’s mother is certified by an edge recording that X is Y’s child. Only one of the three routes attaches evidence at all. Section 8.1 states the exposure that follows and how much of it the committed record can bound, and Section 6.2 measures what the layer changes.

The layer is also nearly free, which matters because a cost argument is sometimes offered in place of an accuracy one. The grounded path adds nothing beyond the single drafting call already required, and at test scale removing the layer altogether saves 2% and 6% of tokens on the two ablation half-splits (Table 3). Whatever the case for or against it, it is

not a cost argument.

4. Experimental Setup

Everything in this section exists to make one inference safe: that a difference between two systems is a difference between architectures. Where something is *not* held constant, we name it here.

4.1. Knowledge Environment

Every dataset this paper consumes is public, standard, and third-party, and all of it arrives through one distribution: the per-question subgraphs published with Reasoning-on-Graphs [7] as `rmanluo/RoG-webqsp` and `rmanluo/RoG-cwq`, built over a Freebase [3] snapshot and used unmodified. What is built here is the environment derived from them: a property graph of 2.59 million entities and 8.31 million triples over 7,058 distinct relation types, the union of every per-question subgraph across both datasets and all three splits. All five systems navigate that one graph, just under two-thirds of whose nodes (63.7%) are unnamed mediators encoding n -ary facts.

An environment assembled this way is friendlier than full Freebase, and we state the consequence rather than leave it to be inferred. Before any system ran, a validation gate measured what fraction of questions have at least one gold answer reachable from a topic entity within the four-hop cap: 97.0% on the WebQSP sample and 99.2% on the ComplexWebQuestions sample. A sample that passed that gate is what the paper calls *certified*. That ceiling is why a failure in Section 7 can be attributed to the system rather than to a missing edge, and equally why **absolute accuracies here do not transfer to a full-Freebase setting**: every comparison in Section 5 is between systems traversing the identical environment, which is what the comparison is for. A second limit concerns expressiveness: the environment carries no typed literal and the tool API no ordering operator, so a question answered by a date, a quantity, or a superlative is unanswerable here by construction.

4.2. Benchmarks, Baselines, and Backbone

We evaluate on WebQSP [9] and ComplexWebQuestions [10], the two standard multi-hop KGQA benchmarks over Freebase. **We sample 400 questions from each** rather

than running the full 1,628 and 3,531 test splits, a pre-specified fallback adopted when the full-split cost was projected at roughly three times the budget available. The sample is stratified by hop count at a fixed seed, so its strata are proportional to the split’s: 256, 128, and 4 questions at one, two, and three or more hops on WebQSP, with 12 whose gold answer the gate found unreachable, and 137, 211, and 49 on ComplexWebQuestions, with 3 unreachable. The resulting 95% bootstrap intervals are roughly ± 4 to ± 5 points on Hits@1. The benchmarks differ less in the typical question than in the tail. Exactly half of the WebQSP sample carries a single gold answer, but its mean is 7.15 gold entities per question and its largest question lists 237, while ComplexWebQuestions averages 1.75 with a median of 1; that tail is one reason we report F1 beside Hits@1.

Four baselines span the design space, three of them controls that bound what a paradigm can do here rather than tuned competitors. A **no-retrieval** control answers from parametric memory alone, and **Vector-RAG** retrieves verbalised triples by embedding similarity. **Static GraphRAG** [4] retrieves a fixed-radius neighbourhood in one shot; its radius is one logical hop and its expansion is capped and unordered (Appendix C.1), so it licenses one narrow claim and no more: *retrieval fixed before reasoning must commit to a radius in advance, and any fixed radius is wrong for some question in the set*. **Think-on-Graph** [5] is the agentic comparator and the one comparison here between two systems of the same paradigm: model-guided beam search over relations and entities, reimplemented on this environment’s tools at the original paper’s beam width and depth of 3 and 3 (Appendix C.2). Reimplementing holds graph, model, budget, and scoring rule constant so that only the algorithm varies, and it also means its numbers are not comparable to the published ones.

Every system shares one frozen backbone, **gpt-5.4-mini-2026-03-17**, a dated snapshot rather than an alias, at temperature 0.0 with reasoning effort disabled, together with the entity resolver, the question files, the output schema, and *one budget meter with the same 25-call cap*; both graph-navigating systems seed from the datasets’ annotated topic entities, which removes entity linking as a confound. Holding all of this constant is what licenses the paper’s central inference: a difference between two systems is attributable to architecture, not to model capacity, retrieval corpus, or spend. Three things are deliberately not held constant. The baselines receive a plain answering prompt, while AGR’s drafting prompt

carries the grounding provisions that are part of the system under test. The two navigators consider candidate sets of different widths, 40 and 20 against 300 and 200. And the shared call cap binds on them at very different rates. The second is where a reader should look first for a confound, and Section 5.3 reports the measurement that bounds it.

4.3. Metrics and Protocol

Accuracy is reported as **Hits@1** and per-question **F1**. All five systems return an unranked set of answer entities, so Hits@1 follows the benchmarks’ convention for that case: a question scores a hit if any returned entity is in the gold set. F1 sits beside it because a system can score a hit on that rule while asserting several wrong entities alongside the right one. A **hedge** is an empty predicted entity set; hedges score zero on both metrics and the hedge rate is its own column, so a system buying precision through abstention is visible rather than flattered. Hits, hedges, and wrong answers partition the whole set, and nothing is dropped from the denominator. Matching is exact after Unicode normalisation, case folding, and whitespace stripping; of AGR’s wrong answers, 1 of 65 on WebQSP and 6 of 99 on ComplexWebQuestions are surface-form near-misses that aggressive normalisation would have scored as hits, so the census of Section 7 is reading real errors. **Groundedness** is measured at two levels, one deterministic over every asserted entity and one a sampled judgement of whether an entity is supported *as the answer* (Appendix C.3). That judge was validated against a human annotator on 100 items, reaching 85% agreement and Cohen’s $\kappa = 0.6995$. That *misses* the 0.70 we pre-specified, by 0.0005, and Section 8 treats the miss as a limitation rather than rounding it away. Efficiency is tokens and model calls, both exact from the shared meter.

The protocol was **pre-specified**: research questions, metrics, baseline certification thresholds, and ablation conditions were fixed and documented before the test sets were constructed, and no hyperparameter was tuned on test. Four decisions were written down before the measurements they govern — the frozen configuration $\alpha = 0.7$ and $\tau = 0.20$ of Section 3, selected on a disjoint 80-question development set; the sample size of 400 questions per dataset; the baseline certification threshold; and the judge threshold of $\kappa \geq 0.7$. The last was missed, and it is reported as missed, because a threshold that counts only when it is met is not a threshold. Nothing was filed with a public registry, so the claim is pre-specification

and not pre-registration. Confidence intervals are 95% percentile bootstrap intervals over questions from 10,000 resamples, and paired comparisons use the exact form of McNemar’s test on per-question correctness. The thirteen prompt templates the five systems send, AGR’s seven and the baselines’ six, are reproduced verbatim in Appendix A, generated from the code by a script so that they cannot drift.

5. Aggregate Results and Where the Margin Lives

The same runs are read several times over: what the five systems answer correctly and what they spend, then where the margin over the agentic baseline comes from, and whether the entities they assert are supported by the graph they navigated. Where AGR is not the best of the five, it says that too.

5.1. Accuracy

Table 1 gives the main matrix. AGR has the highest Hits@1 and F1 in every column, reaching Hits@1 of 0.755 on WebQSP against the agentic baseline’s 0.660 and 0.522 on ComplexWebQuestions against 0.435, so the margin over Think-on-Graph is 9.5 and 8.7 points. The per-question F1 gap is wider still, 0.642 against 0.477 and 0.469 against 0.378, because F1 penalises the extra wrong entities that a set-intersection hit forgives. All eight paired comparisons against the four baselines reject at the conventional level. The two against Think-on-Graph, the only comparisons in which both systems navigate the same graph with the same model under the same call budget, rest on 76 questions AGR alone answers against 38 the other way on WebQSP ($p = 4.8 \times 10^{-4}$), and 86 against 51 on ComplexWebQuestions ($p = 3.5 \times 10^{-3}$). Against RoG’s published figures, measured over the full splits with a backbone fine-tuned on both benchmarks, AGR trails on all four; Appendix D.2 gives that comparison and what it does and does not establish.

Two features cut against a simple reading. **Raw hits mislead on the static graph baseline.** GraphRAG scores 0.340 on WebQSP, below the no-retrieval control’s 0.453, which invites the conclusion that it is broken; the wrong-answer counts invert the picture. The control asserts on 351 of 400 questions and is wrong on 170 of them, an *assertion precision* of 51.6%, where assertion precision is the share of questions a system commits on that it answers correctly. GraphRAG asserts on 177 and is wrong on 41, or 76.8%, because the retrieval

Table 1: Main results, 400 questions per dataset, one frozen backbone and one 25-call budget for every system. Hedge is the proportion of questions answered with an empty entity set. The three single-call systems are controls that bound a paradigm; the comparison the paper reads is the one between the two navigators.

System	WebQSP				ComplexWebQuestions			
	Hits@1	F1	Hedge	Calls	Hits@1	F1	Hedge	Calls
no-retrieval	0.453	0.305	12.2%	1.0	0.307	0.279	38.0%	1.0
Vector-RAG	0.568	0.432	27.0%	1.0	0.203	0.168	48.2%	1.0
GraphRAG	0.340	0.262	55.8%	1.0	0.205	0.183	60.8%	1.0
Think-on-Graph	0.660	0.477	18.8%	12.8	0.435	0.378	22.5%	18.2
AGR	0.755	0.642	8.2%	6.2	0.522	0.469	23.0%	8.9

baselines answer only from the facts retrieved and abstain when retrieval misses while the control guesses from memory. A grounded system can score below an unconstrained one on raw hits while committing four times fewer falsehoods.

And AGR does not lead on every column. On ComplexWebQuestions its hedge rate of 23.0% is marginally *above* Think-on-Graph’s 22.5%, so the accuracy lead is not bought by answering more freely. The two systems commit at effectively the same rate, on 308 and 310 of 400 questions, and from those near-identical commitments AGR is right 209 times and wrong 99, against 174 and 136 for Think-on-Graph: 67.9% assertion precision against 56.1%. It finds 35 more answers and asserts 37 fewer falsehoods than beam search does, from the same number of commitments.

5.2. Cost

AGR reaches those numbers at roughly half the model calls of the agentic baseline, 6.2 against 12.8 on WebQSP and 8.9 against 18.2 on ComplexWebQuestions, ratios of 0.48 and 0.49. The call saving is genuine and the token saving is not: AGR spends 4,511 mean tokens per question on WebQSP against Think-on-Graph’s 3,615, and 6,818 against 5,572 on ComplexWebQuestions. It is the *number of round trips* that halves, not the volume of text, so on tokens Think-on-Graph is a frontier point rather than a dominated one, while on calls it is dominated on both datasets — and calls are what the shared budget meters. Wall-clock time tells the same story in seconds (Appendix D.1). AGR never reaches the

Table 2: AGR against Think-on-Graph, split on whether the shared call cap truncated the baseline. Read from the per-record `budget_exhausted` trace flag.

Subset	n	Think-on-Graph Hits@1	AGR Hits@1
<i>WebQSP</i>			
Think-on-Graph finished	283	0.852	0.788
Think-on-Graph clipped	117	0.197	0.675
<i>ComplexWebQuestions</i>			
Think-on-Graph finished	224	0.629	0.607
Think-on-Graph clipped	176	0.188	0.415

shared call cap: across all 800 test questions it hit the 25-call limit zero times, though its depth cap refuses a deeper expansion on 25.6% of test questions. The baseline reaches the cap far more often.

5.3. Where the Margin Comes From

Beam search costs width times depth, so the shared 25-call cap binds on Think-on-Graph far more often than on AGR. At width 3 a full depth costs the baseline up to thirteen calls (Section 4.2), and the reimplementaion reached its third expansion on 6 of 400 WebQSP questions and 11 of 400 on ComplexWebQuestions, so its published depth of three is nominal under this cap. Splitting the questions by whether the cap cut it off separates two things the aggregate confounds: how good its search is, and whether it can afford to finish. Table 2 gives the split, and it reverses the comparison on one side. Where the budget lets Think-on-Graph run to completion it is *ahead* of AGR, 0.852 against 0.788 on WebQSP and 0.629 against 0.607 on ComplexWebQuestions. The entire aggregate margin comes from the 29% and 44% of questions on which the cap truncates it, where its accuracy collapses to 0.197 and 0.188 while AGR still reaches 0.675 and 0.415.

The split conditions on the baseline’s own outcome, so the reading would be unsafe if the clipped half were simply the hard half, and Table D.6, in Appendix D.7, reads it within hop strata, and every figure the reading rests on is quoted here. On WebQSP the clipped half is not harder for AGR: 81 of the 117 clipped questions are single-hop, AGR scores 0.728 on them against 0.766 on the single-hop questions the baseline finished, and the baseline’s fall from 0.846 to 0.160 across the same two sets is the cap, not the questions.

On ComplexWebQuestions the halves do differ — AGR scores 0.309 on the clipped single-hop questions against 0.609 on the finished ones — so there truncation and difficulty are confounded and the affordability reading is weaker. The finished halves are not uniform either: on ComplexWebQuestions the baseline leads AGR only on single-hop questions, 0.754 against 0.609, and trails on two-hop, 0.561 against 0.610, which is the planner asymmetry of Section 6.1 seen from the other side.

The correct reading is affordability, not resolution quality. Exhaustive beam search finds answers well when allowed to finish; guided search finds them almost as well and finishes within a budget the beam search cannot. On a fixed spend the second is the better architecture; the first would win if spend were unconstrained. The cap of 25 calls is itself a choice, and a sweep of it is the first experiment Section 8.4 names.

One asymmetry bounds how far that reading extends, and it is the candidate-width difference named in Section 4.2. Both navigators call the same tools but truncate what those tools return at different widths: Think-on-Graph keeps the first 40 relations per entity and the first 20 neighbours per relation, against AGR’s 300 and 200. The relation list is sorted by *descending* fanout, so a binding cut discards the low-fanout tail, which is where the relation answering a specific question tends to sit. Over the committed tool logs the 40-relation cut binds on 31.6% of the 1,651 entities Think-on-Graph expanded and the 20-neighbour cut on 32.8% of its 7,992 neighbour calls, against 1 of AGR’s 3,097 expanded entities and 3.3% of its neighbour calls.

The affordability conclusion survives this, because a thinner candidate pool cannot explain why a system runs *out of calls*. What it bounds is the other half of the split: the 0.852 and 0.629 the baseline reaches where it finishes were measured over a narrower pool than AGR’s, so they are a **lower bound** on what an equal-width reimplementation would reach rather than an estimate of it, and since the baseline is already ahead there, equalising the widths would be expected to widen that lead. Equalising them is the first change we would make in a follow-up. That re-run would not vary one thing, though: wider candidate sets make each pruning call dearer, so equal-width pruning would also raise the baseline’s clip rate and move the boundary of the very split it would be read from. The two effects have to be read apart, which means reporting that split at each setting rather than

an aggregate in which they cancel or compound invisibly. Appendix D.10 adds the check on whether AGR buys hits by naming more entities; it does not.

5.4. Groundedness

Across both datasets AGR asserted 1,709 entities, and **zero of them were ungrounded**: every entity it named exists in the environment and is reachable from a topic entity within the hop cap, while the parametric control asserted 1,001 entities of which 22.1% failed that test. Two qualifications come first. The property is close to guaranteed by construction for any system that copies its entities out of tool results. And **it belongs to graph navigation, not to the verification layer**: Think-on-Graph also reaches exactly zero ungrounded assertions, on 1,265 entities, without any claim-checking component whatsoever.

A second tier asks whether an asserted entity is supported *as the answer*, and it separates the systems far less; Appendix D.4 gives every cell, and no between-system gap there survives $n = 60$. The level is the useful finding rather than the ranking. **Zero structural hallucination coexists with between a third and nearly two-thirds of assertions failing a strict semantic-support test**, for every system, including the two that are structurally perfect. Semantic support is hard for the paradigm, not for one system within it.

6. Component-Level Attribution

Four conditions each remove exactly one component from the unmodified system, as Appendix B.6 specifies, and re-run the identical questions: no planner, no backtracking, no claim verification, and embedding-only scoring ($\alpha = 1$). Each is compared against the reference on the same half-split, drawn as half of every hop stratum rounded down, which gives $n = 200$ on WebQSP and $n = 198$ on ComplexWebQuestions, with McNemar’s test on the paired per-question outcomes. The reference row is the unmodified system restricted to the same questions, read from the main-run records rather than re-run, so it is provably the system that produced Table 1. **Throughout this section a delta is ablated minus reference, so a positive number means removing the component *improved* the metric**. The p -values in Table 3 are reported as computed; a correction for multiple

Table 3: Component ablations. $\Delta F1$ is ablated minus reference: positive means the system got better without the component. The interval is a paired percentile bootstrap over questions (10,000 resamples); p from McNemar’s exact test on paired per-question correctness.

Removed	WebQSP ($n = 200$)				CWQ ($n = 198$)			
	$\Delta F1$	95% CI	p	Tokens	$\Delta F1$	95% CI	p	Tokens
Planner	+0.083	[+0.038, +0.130]	0.006	−31%	−0.047	[−0.108, +0.013]	0.088	−21%
Backtracking	−0.013	[−0.038, +0.010]	0.727	−7%	0.000	[−0.031, +0.030]	1.000	−18%
Verification	+0.004	[−0.006, +0.018]	1.000	−2%	−0.009	[−0.024, +0.000]	1.000	−6%
Model scoring	−0.016	[−0.060, +0.026]	0.664	−25%	−0.008	[−0.046, +0.030]	0.481	−31%

comparisons was not among the pre-specified decisions, and applying one after seeing the results would be a choice made on the results, so the four tests are read as descriptive.

6.1. When Planning Hurts

One component produced a detectable effect, and it produced it in the wrong direction. **Removing the planner improves WebQSP F1 by 0.083** ($p = 0.006$) while cutting token use by 31%. Twenty-one questions were answered correctly only without the planner, against six correct only with it, so the cheaper system is the more accurate one.

The mechanism is over-decomposition. WebQSP is predominantly a single-hop benchmark: 256 of its 400 questions need one hop. Asked to decompose a question with no compositional structure, the planner manufactures sub-objectives that are not really separable, the explorer then scores candidate edges against a fragment of the question rather than the question, and it discards exactly the context that would have identified the right relation. The planner prompt carries a single-hop exemplar to show that not decomposing is a valid output, and that is evidently not enough. The per-stratum breakdown locates the damage: on the WebQSP half-split, removing the planner raises one-hop Hits@1 from 0.75 to 0.85 and two-hop Hits@1 from 0.84 to 0.89.

On ComplexWebQuestions the direction reverses, as the mechanism predicts. That benchmark is genuinely compositional, only 137 of 400 questions are single-hop, and removing the planner costs 0.047 F1. **That arm does not reach significance** ($p = 0.088$), and we report it as a direction rather than an effect; we do not call it marginally significant, because at this sample size that phrase would assert more than the data carries. The strata show a coherent mechanism rather than noise: without the planner, one-hop Hits@1

improves from 0.49 to 0.54, two-hop collapses from 0.48 to 0.34, and three-hop-plus moves from 0.62 to 0.54. That is the pattern in which decomposition helps genuine composition and hurts questions that do not need it, and it is the pattern Table D.6 shows from the other side. Pooling the two datasets would average a significant improvement against a non-significant harm and report approximately nothing. The asymmetry *is* the result: decomposition is not a general-purpose benefit but a mechanism whose value depends on whether the question has compositional structure to find.

6.2. Three Components with No Detected Effect

The remaining three conditions moved nothing detectable, and we report them at the same length as the one that did. A field that reports only what moved cannot attribute anything.

Backtracking. Removing it changes WebQSP F1 by -0.013 ($p = 0.727$) and ComplexWebQuestions F1 by 0.000 ($p = 1.000$), while saving 7% and 18% of tokens. The mechanism fires rarely: the reference condition records 38 and 108 backtracks over the 398 paired questions, and on those same runs the meter refuses a further attempt on 28 of them, 7.0%. A component this seldom exercised cannot show a large effect. One implementation detail bounds the null further and is recorded rather than smoothed over: the backtracker bans the edges of the most recent pass but restores an often older snapshot, so the null describes backtracking as implemented rather than as specified (Appendix D.9). The accurate summary is that the search rarely needs to undo a decision, not that undoing is worthless.

Model scoring. Setting $\alpha = 1$ removes the language model from edge selection entirely. F1 falls by 0.016 and 0.008, neither significant, while tokens drop 25% and 31%. The model term is the single most expensive part of the navigation loop, and this experiment cannot establish that it earns its cost. The development-set sweep that froze α points the same way at a smaller scale, and one confound goes with the condition; Appendix D.9 gives both.

Claim verification. Removing the checking routes of the layer AGR was designed around, while leaving the drafting call unchanged, changes WebQSP F1 by $+0.004$ and ComplexWebQuestions F1 by -0.009 , $p = 1.000$ on both. Across the half-splits — 200 questions on WebQSP and 198 on ComplexWebQuestions — the two conditions disagreed on exactly one question each. **Claim verification does not measurably change how**

often AGR is right. The result is expected rather than disappointing, for the reason Section 3.2 gave before any of these numbers appeared: on most questions the layer is a pass-through certifying a draft that was already grounded. On test it asks for a repair on 52 of 800 questions, exploration actually resumes on 28, and 13 end grounded that did not start so. A mechanism that alters the output on a few per cent of questions cannot move an aggregate accuracy metric, and an ablation is the right instrument for establishing that rather than assuming it either way. What the layer delivers is therefore not an accuracy gain but the output contract: an answer paired with the traversed triples that support it, and an entity list that is a deterministic function of the verdicts rather than a second generation. That is an auditability claim, not an accuracy claim, and Section 8.1 states how far it reaches; Section 5.4 has already shown that the layer cannot claim credit for groundedness.

6.3. Statistical Power

These are nulls at low power, not demonstrations of absence. McNemar’s exact test sees only the *discordant* pairs, and the null conditions produced very few. Removing claim verification changed the outcome on exactly one question per dataset, and with a single discordant pair no split whatsoever can reach significance, so that row is not a test that failed to reject: it is the two conditions agreeing on 396 of the 398 paired questions, a statement about how rarely the layer alters an answer rather than about statistical power. Backtracking produced 8 and 11 discordant pairs, and model scoring 21 and 18. At those counts the smallest imbalance the exact test could have called significant is 8 and 9 questions for backtracking, meaning all eight would have had to fall the same way, and 11 and 10 for model scoring, which is between 4.0 and 5.5 points of accuracy. Differences below that were undetectable at any power, whatever their true size.

For calibration: detecting a true 2:1 discordance ratio at 80% power requires roughly 72 discordant pairs, and the largest of these conditions produced 21, at which count the smallest ratio detectable at 80% power is nearer 4:1, so a true 2:1 effect would have been caught about a quarter of the time. **Three of four components show no effect at this power;** none is shown to have no effect. One more count bounds what any single removal could have done: the union of the four discordant sets holds 42 of the 200 WebQSP questions and 55 of the 198 on ComplexWebQuestions. On the remaining 158 and 143, no condition changed

the outcome. The intervals in Table 3 say the same thing in F1, since the planner’s WebQSP interval is the only one of the eight that does not contain zero. Each condition was also run once and the components were removed one at a time, so three nulls are consistent with the three being individually dispensable and jointly necessary, and equally with a system that has none of them matching this one at a fraction of the tokens; the condition that separates those readings was not run, and Section 8.4 names it.

7. Error Analysis

After the label-defect exclusions of Section 7.2, AGR’s residual non-hits on the two samples number 262: 86 on WebQSP and 176 on ComplexWebQuestions. Six of them, one and five, are the surface-form near-misses of Section 4.3, scoring artefacts rather than reasoning failures, and were set aside. Every one of the remaining 256, 85 and 171, was read and labelled by hand against its full execution trace, so the counts are over a population rather than estimates from a sample. The reading protocol was fixed before it began: for each trace, find where the trajectory *first* left the correct path, and label that point rather than the last visible symptom. Wrong answers and hedges are kept separate, because a wrong answer is a reasoning error while a hedge is usually a coverage gap that never produced a committal claim. Figure D.2, in Appendix D.8, gives the census by category. Its largest categories are unsurprising — wrong relation choice is the base rate of graph navigation, and composite claims dominate ComplexWebQuestions, whose template-generated questions carry conjunctions that WebQSP’s search-log questions rarely do. Questions whose answer is a date, a number, or a superlative fall into the class Section 4.1 named as unanswerable here, and on ComplexWebQuestions the system hedges on them more than three times as often as it asserts something wrong, 24 against 7. Two smaller categories are reported in full instead, because they generalise past this architecture.

7.1. The Echo Attractor

The most instructive failure mode is one that every grounding check in this paper passes. The system resolves a real entity, reached through a real edge, and states a fact that is genuinely true. It is not what was asked. We name this the **echo attractor**: a true, well-grounded entity sitting one hop from the answer in the graph’s structure. It takes

two shapes. *Topic confusion* picks a neighbour in a tight naming cluster, answering that John Glover plays Lionel Luthor when asked who plays Lex Luthor. *Granularity* picks the right entity family at the wrong level, answering “Kingdom of Denmark” where the gold answer is “Denmark”, a genuinely distinct node. A reader may prefer to call the second a label-granularity problem; we count it against the system because the two nodes are distinct in the graph and the question’s gold names one of them, and the label-defect pass of Section 7.2 is where the other reading was tested.

What makes this a mechanism rather than a residue category is that nothing went wrong in the parts of the system built to prevent things going wrong. The right edge exists, is retrieved, and is walked; the entity returned is present in the environment, and every structural check certifies it. The failure is entirely in *which* true fact gets surfaced, which a claim-level check against traversed adjacency cannot catch by construction, because the claim is true.

The consequence for evaluation methodology is larger than the consequence for this system. Systems that share a graph and a backbone converge on the *same* wrong entity for the *same* questions, and Section 7.2 measures how often, so any protocol treating systems as independent observers cannot see it. Consensus filtering, ensembling, and majority-vote gold repair all take agreement as evidence of correctness, and here agreement is evidence of a shared bias instead, so such a protocol will confidently ratify the wrong answer in exactly the cases it was introduced to catch. The echo attractor accounts for 13 of the 256 failures read. The rate is small and the sample does not support a precise one: the contribution is the named mechanism and its methodological consequence, not its frequency at this n .

7.2. Benchmark Label Defects

The census required a second pass, because a failure against a wrong gold label is not a failure of the system. A consensus pre-pass flagged every question where at least three of the five systems agreed on an answer the benchmark scored wrong, and each was then adjudicated individually against the graph and the question text. The two counts must be kept apart. The pre-pass **flagged** 58 WebQSP and 47 ComplexWebQuestions questions; adjudication **confirmed** a genuine label defect in 22 and 19 of them, which is 5.5% and

4.8% of each sample, splitting into gold answers that are simply wrong (12 and 17) and questions whose phrasing admits several defensible answers of which the benchmark lists one (10 and 2). All 41 were removed before the census began, as benchmark artifacts rather than system failures. The hand-read then found 16 more that the pre-pass had missed — 3 on WebQSP and 13 on ComplexWebQuestions — so **57 distinct questions** across the two benchmarks turned out to need correcting. That the census recovered defects a dedicated consensus pass had missed is itself an argument for reading failures individually rather than screening them in aggregate.

The gap between flagged and confirmed is not adjudication noise. The pass flagged 105 questions and adjudication confirmed 41, so roughly three-fifths of what it flagged was several systems converging on the same wrong entity. Table D.5, in Appendix D.5, reads those 64 cleared questions row by row; the triage filed 52 of the 85 cleared rows as the echo mechanism of Section 7.1 and the rest as relation confusion, an ignored constraint, or the environment gap. Those rows also show that the agreeing systems are not independent observers for a second reason besides the graph: they share one backbone and one resolver, and the no-retrieval control, which sees no graph at all, joins the majority of the cleared ComplexWebQuestions rows (Appendix D.11). Convergence there is at least as much a property of what the backbone believes as of the graph’s neighbourhood structure, and the methodological consequence does not depend on which cause dominates: had we accepted the pre-pass without per-item adjudication, we would have “corrected” the benchmark toward the systems’ shared bias and reported an inflated accuracy for every system, including our own.

The accuracy tables are not rescored. Section 5 reports every system exactly as measured over the full samples. Rescoring against labels we adjudicated ourselves would substitute one contestable ground truth for another, and the defects cut both ways, so removing the adjudicated questions moves every system by at most 0.020 Hits@1 and changes no ranking (Appendix D.13). The adjudication was made by one author with the consensus signal visible, and Section 8.3 carries that as the limitation it is.

8. Discussion and Limitations

The limits of this study bound its claims, and we state all of them rather than the flattering subset.

8.1. *What Bounds the Instrument*

Wrongful acceptance is unmeasured, and it is the most serious gap in this work. The verifier logs the claims it rejects and not the ones it accepts, so a claim certified on an edge unrelated to the relation it asserts leaves no trace. The exposure is real rather than hypothetical, because both structural routes ignore the relation and the direction: any edge between two entities certifies any claimed relation between them. Nothing in the committed record says how often that happened, and Appendix D.12 gives the interval, two orders of magnitude wide, that is all the record supports. The same gap means the layer has no measured precision or recall as a classifier of claims, which is the first number a verification component should carry.

The output contract cannot be audited from the run records. AGR emits an answer paired with the traversed triples supporting it, and the pairing holds inside the run, but the logger writes the *count* of supporting triples and discards the list, so no committed artifact in this work contains a single supporting triple. Persisting each accepted claim with its route and its matching triples is one logging change, and it would repair this limitation and the previous one together.

Structural grounding is not answer adequacy. Both navigating systems assert zero ungrounded entities, which under the definition used is close to guaranteed for any system that copies entities out of tool results. It says their answers are made of the graph’s own material; it does not say they answer the question. The echo attractor of Section 7.1 is the counter-example, a wrong answer that passes every check this paper applies.

8.2. *What Bounds the Comparisons*

Sample size, and one budget. Every result rests on 400 questions per dataset rather than the full splits, giving 95% bootstrap intervals of roughly ± 4 to ± 5 points on Hits@1. The main accuracy margins clear that width, at 9.5 and 8.7 points, as does the planner effect, and the groundedness contrast at zero against 22.1% is not close; the semantic-tier gaps, the

hedge-rate difference on ComplexWebQuestions and the last step of the rising hop-depth curve are within noise and reported as such. The 25-call cap was likewise pre-specified but is still a choice, and Section 5.3 shows the entire margin over the agentic baseline living on the questions where it truncates. A cap sweep would move that boundary; it was not run.

An identical environment is not identical access to it. The agentic baseline truncates candidate sets at 40 relations and 20 neighbours against AGR’s 300 and 200, a cut binding on about a third of its expansions and effectively none of ours (Section 5.3), which is why its figures where it finishes are a lower bound rather than an estimate. It is also the only agentic comparator: Plan-on-Graph, whose two mechanisms AGR’s planner and backtracker re-host (Section 2), was not run, so the planner result speaks to decomposition as implemented here and to Plan-on-Graph’s by analogy only. **The static graph baseline is radius-bounded** in turn, so its decay across hop strata measures the radius as much as the paradigm, and Appendix D.3 declines to offer it as evidence for either.

One backbone, and nondeterminism. All five systems run on gpt-5.4-mini, frozen and closed, so this paper establishes how these architectures compare *on one backbone*, not how the comparison scales across model capacities. Temperature-zero decoding against a hosted API is greedy rather than deterministic, and measured trajectory stability is about 67%. Each system and each ablation condition was run once and no seeds were averaged, so the intervals and paired tests bound the sampling variance of the question draw and nothing else. Large defended margins and per-question pairing make a rank reversal from decoding noise unlikely, but unlikely is the strongest statement available: the repeated matrix that would bound the variance was affordable at the observed cost and was not run, which is a gap in the design rather than a budget constraint.

Ablation power. The four ablations run on half-splits of $n = 200$ and $n = 198$, and the paired test sees only the questions on which two conditions disagree, as few as one for claim verification. Three of four components show *no effect detected at this power*, which is not the same as no effect, and Section 6.3 gives the smallest difference each could have detected. The components were also removed one at a time, so the three nulls do not say whether a system with none of them would match this one.

8.3. What Bounds the Measurements

The environment is friendlier than the knowledge base it stands in for, and its node identity is coarser. Per-question subgraphs with a 97%-plus answer-reachability ceiling are a more forgiving substrate than full Freebase: between-system comparisons hold, because all five systems navigate the identical graph, but absolute accuracies do not transfer, and Appendix D.2 shows the gap to a published system trained on the benchmarks. Topic entities are **given, not linked**, which isolates navigation from entity linking as intended but means every reported accuracy assumes a linking step a deployed system would have to perform. And entities are keyed by name, so **homonyms merge**: distinct real-world entities sharing a surface form collapse to one node, which can manufacture a path between things not actually connected and can equally destroy a distinction a question depends on. Neither direction is measured.

The measured accuracy has an unmeasured floor. An extraction defect converts correct reasoning into a wrong entity list on one question shape: on “list the professions” and “what did X do” questions the drafted text names every gold value and the scored entity list collapses to the sentence’s subject. Accuracy on that shape is a lower bound rather than an estimate.

The judge missed its own bar, and the adjudication was single-annotator. Agreement between the semantic-tier judge and a human annotator reached $\kappa = 0.6995$ against a pre-specified 0.70 — a miss, by 0.0005, which is why those numbers are reported as corroborating rather than decisive and why no conclusion rests on them alone. The judge also runs on the same backbone as the systems it judges. Separately, the benchmark-defect exclusions and the failure census rest on single-annotator judgements made by an author with the consensus signal visible (Section 7), and one development-set relabelling was made after its outcome was known. It is flagged in place, and no aggregate was rescored.

Scope. English factoid questions over a static graph, with no temporal facts, no graph construction, and an offline prototype. The environment carries no typed literals and the tool API no ordering operator, so questions answered by a date, a quantity, or a superlative are out of reach by construction (Section 4.1). Wall-clock figures are magnitudes with a cache caveat (Appendix D.1), not a production-readiness claim.

8.4. Outlook

The findings fix the order of the next experiments, and each is cheap against the cost of the study it would complete: the agentic baseline at AGR’s candidate widths under a sweep of the call cap, replication with the joint-removal condition Section 6.3 names, the full test splits, and one logging change that would convert the two limitations opening this section into measurable quantities. Appendix D.6 says what each would settle.

The most actionable finding points forward rather than back. If decomposition helps only when a question has compositional structure, improving accuracy by 0.083 F1 where that structure is absent and costing 0.047 where it is present, then the planner should not run unconditionally: **gate decomposition on question structure**. A hop-count predictor gating the planner would capture both arms of that asymmetry, and it can be evaluated with the instrument used here. More generally, agentic components should be treated as conditionally rather than universally beneficial, and the condition is worth measuring.

9. Conclusion

We built an agentic knowledge-graph question-answering system whose four components can be removed one at a time, ran it against four baselines on one frozen backbone under one call budget, and took it apart. The aggregate result is the one the field usually reports: the agentic system has the highest accuracy in every column, at roughly half the model calls of the agentic baseline. The rest of the paper is what that result turns out to be made of. The entire margin over the agentic baseline comes from the questions on which the shared budget truncates it, so it is a finding about affordability under a fixed spend rather than about reasoning power. Of four components, three could not be shown to earn their cost at the power available — including the claim-verification layer the system was designed around, which changes the answer on one question per dataset and delivers an output contract rather than an accuracy gain. The fourth was actively harmful on the shallower benchmark: removing the planner improved accuracy and cut token use, because decomposing a question that has no compositional structure discards the context that would have answered it.

The shape of the residue matters as much as the margin. A system that traverses a real

graph and copies its answers out of real triples can be made to assert nothing ungrounded, and the navigation paradigm achieves that, not any checking layer built on top. What remains is an answer that is grounded, connected, and true, and is still not the answer to the question that was asked; systems sharing a graph and a backbone converge on that answer together, so a protocol that takes their agreement as evidence ratifies it. Fact verification against a knowledge graph turns out to be the easier half of the problem, and relevance verification is the half still open. The attribution result is finally a claim about how this field measures rather than about this architecture: a component adopted because the assembly containing it performed well may be contributing nothing, and a margin reported as an architectural advantage may be a budget regime. The only way to know is to remove the component, vary the budget, and test.

Appendix A. Prompt templates

Seven templates constitute the whole of AGR’s interface to the language model, and six serve the baselines: one for the parametric control, one shared by the two static retrieval baselines, and four belonging to the reimplemented agentic baseline. All thirteen are reproduced here verbatim from the source that sends them, in the order a question encounters them, by a script that reads the code (`scripts/build_paper_appendix.py` in the repository named in the data availability statement). No template carries a configuration constant, so one cached response serves every condition of a sweep; budgets are counters in code rather than instructions, and the evaluator’s remaining depth and backtracks are the only budget figures any prompt carries. Every call asks for JSON against a schema stated in the code beside the template.

Appendix A.1. Planner

`agr/planner.py`, constant `PLANNER_PROMPT`. Sent once per question unless the planner is removed. The first exemplar is the single-hop case that is not decomposed. Braces written `{like this}` are format fields filled at send time; doubled braces are literal.

You decompose questions into a minimal ordered chain of sub-objectives for navigating a knowledge graph. Each sub-objective must be answerable by following one relation (possibly through an event node). Use `#N` to reference the result of sub-objective `N`. Use as FEW sub-objectives as possible.

For topic_mentions, extract ONLY specific named entities that appear literally in the question (people, places, organizations, works, products). Do NOT extract generic type words such as "celebrity", "university", "country", "senator", "airport", or "currency" -- these describe what kind of answer is wanted, not where to start searching.

Example 1 (single hop -- do NOT decompose):

Question: "who is the president of France?"
{ "sub_objectives": ["find the president of France"],
 "topic_mentions": ["France"] }

Example 2 (composition):

Question: "who directed the film that won Best Picture in 1998?"
{ "sub_objectives": ["find the film that won Best Picture in 1998",
 "find the director of #1"],
 "topic_mentions": ["Best Picture"] }

Example 3 (conjunction):

Question: "which countries border France and have Spanish as official language?"
{ "sub_objectives": ["find countries that border France",
 "find which of #1 have Spanish as an official language"],
 "topic_mentions": ["France", "Spanish"] }

Example 4 (superlative):

Question: "what is the largest city in the country where the Nile ends?"
{ "sub_objectives": ["find the country where the Nile ends",
 "find the largest city in #1"],
 "topic_mentions": ["Nile"] }

Question: "{question}"

Appendix A.2. Relation scorer

agr/scorer.py, constant SCORE_PROMPT. One batched call per explorer pass over the twenty candidates with the highest embedding score. Not sent when $\alpha = 1$. Braces written {like this} are format fields filled at send time; doubled braces are literal.

Sub-objective: {objective}

Candidate knowledge-graph edges (anchor entity --relation-->):
{candidates}

For EACH candidate, rate from 0.0 to 1.0 how likely following that edge from that anchor leads toward completing the sub-objective. 1.0 = directly answers it, 0.0 = irrelevant. Judge the RELATION MEANING in context of the anchor, not surface word overlap.

Appendix A.3. Evaluator

agr/nodes.py, constant EVAL_PROMPT. One call per explorer pass. The remaining depth and backtracks are substituted at send time; no other prompt carries a budget figure. Braces

written `{like this}` are format fields filled at send time; doubled braces are literal.

```
Current sub-objective: {objective}
Recently retrieved facts:
{facts}
Remaining budget: {d_left} more expansions, {b_left} more backtracks.

Decide:
- "answer" if the facts complete the sub-objective (list satisfying entities in "
resolved", exactly as named in the facts),
- "continue" if this direction is promising but incomplete,
- "backtrack" if this direction is a dead end.
```

Appendix A.4. Drafter and claim decomposer

`agr/nodes.py`, constant `DRAFT_PROMPT`. One call when the search ends. Produces the prose, the answer entities, and the atomic claims the verifier checks. Braces written `{like this}` are format fields filled at send time; doubled braces are literal.

```
Question: {question}
Facts retrieved from the knowledge graph:
{facts}
Entities identified as promising during exploration:
{candidates}

1. Draft an answer using ONLY the facts and candidates above. Use entity
   names exactly as written. If insufficient, say what could not be
   determined.
2. List the answer entities themselves in "answer_entities", copied EXACTLY
   as they are named in the facts above -- these are the entities your draft
   asserts as the answer, not every entity it mentions. If your draft is a
   hedge ("could not be determined"), "answer_entities" MUST be [].
   IMPORTANT: "answer_entities" must be specific entities of the kind the
   question asks for (a person for "who", a place for "where", etc.).
   Never restate the question or name entity types/categories as the
   answer. If you cannot name specific answer entities from the facts,
   your draft must be a hedge and "answer_entities" must be [].
3. Decompose your draft into atomic factual claims, each as
   {"h": entity name, "r": short relation phrase, "t": entity name}}.
   Only include claims your draft actually asserts. A hedged draft has zero
   claims.
```

Appendix A.5. Entailment check

`agr/nodes.py`, constant `ENTAIL_PROMPT`. One batched call over the claims neither structural route settled. Braces written `{like this}` are format fields filled at send time; doubled braces are literal.

```
Retrieved knowledge-graph triples:
{facts}
```

For each claim below, answer whether it is directly supported by the triples above (true) or not (false). Unsupported includes: plausible but absent, contradicted, or only inferable via outside knowledge.

```
Claims:
{claims}
```

Appendix A.6. Rewriter

agr/nodes.py, constant REWRITE_PROMPT. One call on the give-up path only. The entity list it is given is already filtered; its own opinion about entities is discarded. Braces written {like this} are format fields filled at send time; doubled braces are literal.

```
Question: {question}
Draft answer: {draft}
The following claims in the draft are NOT supported by the knowledge graph:
{unsupported}
The following entities remain verified and MUST be kept in the answer,
named exactly as written: {kept}

Rewrite the draft: keep the verified entities and their supported claims,
and remove or explicitly hedge the unsupported claims. If the kept list is
empty, state that the answer could not be verified against the knowledge
graph. Respond with prose only.
```

Appendix A.7. Answerer (verifier skipped)

agr/nodes.py, constant ANSWER_PROMPT. Reached only when the drafting call itself failed; no committed run took this path. Braces written {like this} are format fields filled at send time; doubled braces are literal.

```
Question: {question}
Facts retrieved from the knowledge graph:
{facts}

Entities identified as promising during exploration:
{candidates}

Answer using ONLY these facts and candidates. Give the answer entity name(s)
exactly as they appear above. If the information is insufficient, state what
could not be determined.
Also list in "answer_entities" the entities your answer
asserts (exactly as named above); [] if it asserts nothing.
```

Appendix A.8. No-retrieval control

agr/baselines/noretrieval.py, constant PROMPT. The single call of the parametric control.

Braces written {like this} are format fields filled at send time; doubled braces are literal.

```
Answer this question with the name(s) of the answer entity or
entities. Be concise. If you do not know, say so and return [].
```

```
Question: {question}
```

Appendix A.9. Vector-RAG and static GraphRAG

agr/baselines/vectorrag.py, constant PROMPT. Shared by the two static retrieval baselines, which differ only in what they retrieve. Braces written {like this} are format fields filled at send time; doubled braces are literal.

```
Facts (retrieved from a knowledge graph):
{facts}
```

```
Question: {question}
```

```
Answer using ONLY the facts above. Name entities exactly as written in the
facts. If the facts are insufficient, say so and return [].
```

Appendix A.10. Think-on-Graph: relation pruning

agr/baselines/tog.py, constant REL_PRUNE. One call per frontier entity per depth, over the first 40 relations returned. Braces written {like this} are format fields filled at send time; doubled braces are literal.

```
Question: {question}
Current entity: {entity}
Available relations (relation, direction, fanout):
{relations}
Select up to {w} relations most likely to lead to the answer.
Return {"relations": [{"rel": "...", "dir": "in|out"}]}.
```

Appendix A.11. Think-on-Graph: entity pruning

agr/baselines/tog.py, constant ENT_PRUNE. One call per selected relation per depth, over the first 20 neighbours returned. Braces written {like this} are format fields filled at send time; doubled braces are literal.

```
Question: {question}
Candidate entities reached (via {entity} --{rel}-->):
{entities}
Select up to {w} entities most likely on the path to the answer.
Return {"entities": ["..."]}.
```

Appendix A.12. Think-on-Graph: sufficiency

agr/baselines/tog.py, constant SUFFICIENT. One call per depth, on the last sixty gathered triples. Braces written {like this} are format fields filled at send time; doubled braces are literal.

```
Question: {question}
Knowledge triples gathered so far:
{triples}
Can the question be answered from these triples alone?
Return {"sufficient": true|false}.
```

Appendix A.13. Think-on-Graph: answer

agr/baselines/tog.py, constant ANSWER. One call when the search stops, whether it finished or hit the cap. Braces written {like this} are format fields filled at send time; doubled braces are literal.

```
Question: {question}
Knowledge triples:
{triples}
Answer using ONLY these triples; name entities exactly as written.
If insufficient, say so and return [].
```

Appendix B. The instrument in detail

Section 3 gives the loop at the width the attribution study needs. This appendix records the mechanics a reimplementation would want, in the order a question meets them.

Appendix B.1. Graph tool API

Four graph operations back the loop, as parameterised Cypher templates against a Neo4j property graph rather than free-form queries the model would write itself, because a generated-query failure is indistinguishable in the results from a reasoning failure. `search_entity` resolves a surface form to a node through three tiers, exact name, then full-text index, then vector similarity, and stops at the first that returns a match. `get_relations` returns the relation types incident on an entity with their direction and fanout, after a blocklist removes schema relations, sorted by descending fanout and cut at 300 rows. `get_neighbors` returns at most 200 neighbours along one relation. Unnamed mediator nodes, which encode n -ary facts and make up 63.7% of the graph (Section 4.1), are traversed transparently, so

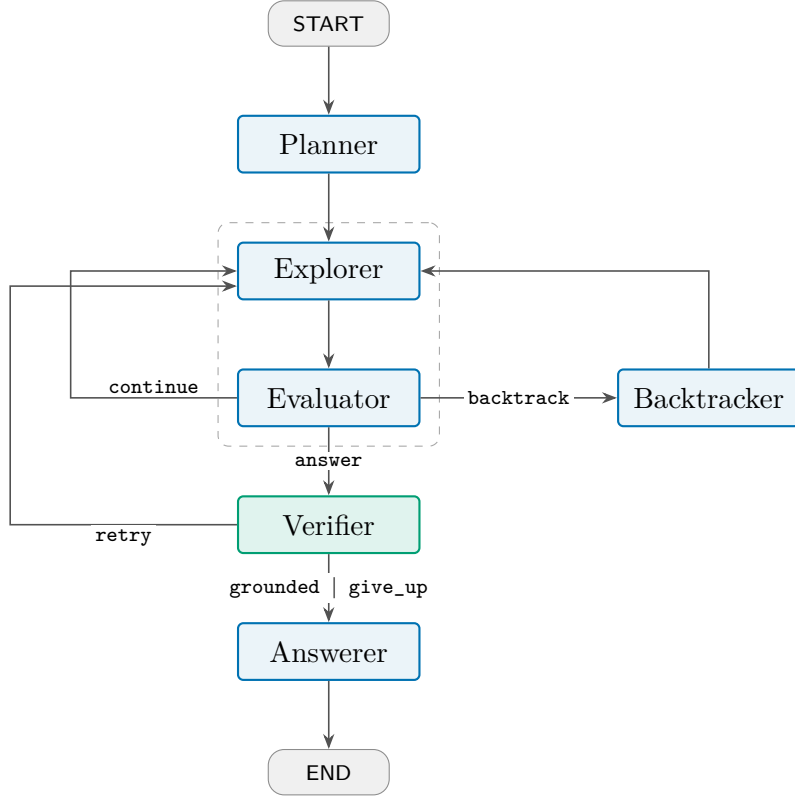


Figure B.1: The AGR state machine. The dashed region is the Explorer $\leftrightarrow$ Evaluator search loop. Evaluator $\rightarrow$ Backtracker $\rightarrow$ Explorer restores an earlier frontier, and Verifier $\rightarrow$ Explorer is verification-driven re-exploration. The budgets of Appendix B.5 bound all three cycles. Every path from Evaluator to Answerer passes through the Verifier.

a mediator path costs one hop. `verify_connection` asks whether two entities are adjacent directly or through one mediator, ignoring the relation. A fifth operation, `verify_triple`, was built first to check a claim $\langle h, r, t \rangle$ by asking whether that exact relation held between the two entities. It did not survive contact with generated text, because claim decomposition phrases relations in English (“played for”, “is the capital of”) and those phrases rarely correspond to a single Freebase predicate, so matching on them rejected claims that were true. Nothing in the running loop calls it.

Appendix B.2. Edge scoring

The explorer gathers every relation row of every anchor into one candidate set, at most five anchors, and scores it in one pass. Each row c , an anchor together with a relation and

a direction, receives

$$\text{score}(c) = \alpha \cos(v_{\text{rel}(c)}, v_o) + (1 - \alpha) \text{LLM}(c), \quad (\text{B.1})$$

which blends two signals at the frozen $\alpha = 0.7$. The first is the cosine similarity between the candidate relation’s embedding and the embedding of the active sub-objective o , both from **all-MiniLM-L6-v2** [22], with the relation vocabulary of 7,058 types embedded once in advance. The objective, not the original question, is what gets embedded, and that substitution is the concrete mechanism through which decomposition is meant to help. The second is a batched model judgement, one call, over the twenty candidates with the highest embedding score; every other candidate receives zero for that term. The blend is applied after the model call rather than inside the prompt, so that a prompt free of any configuration value lets an α sweep reuse one set of cached responses across every condition. The three best rows per anchor, skipping banned ones, are expanded, and the five neighbours with the highest reaching score become the next frontier.

Appendix B.3. Evaluation, routing, and backtracking

The evaluator is one model call. It sees the active sub-objective, the thirty traversed triples most similar to it by the same embedding, and the remaining depth and backtrack budget, and it returns one of three decisions together with the entities it considers to satisfy the objective. Those entities are accepted only if they are names in the traversed set, so the evaluator cannot introduce an entity the graph did not return. When it marks an objective done, its resolved entities become the anchors for the next objective.

The router after it applies three tests in order. If the depth or the wall-clock budget is spent, the loop ends and the verifier runs. Otherwise, if a backtrack trigger fires and the backtrack budget allows, the backtracker runs; three triggers exist, namely that the last expansion added no triple, that the best score of the last expansion fell below $\tau = 0.20$, or that the evaluator asked to backtrack. Otherwise the evaluator’s decision stands. The backtracker pops the snapshot with the highest recorded score from the stack, which is often not the most recent one, restores its anchors and depth, and bans the anchor–relation–direction rows the most recent expansion followed. The mismatch between the snapshot it restores and the rows it bans is deliberate in neither direction, and Section 6.2 measures what

it costs. This is where the mechanics part from Plan-on-Graph [6], whose decomposition and reflection live in prompts the model executes: here the plan is a data structure the program advances, two of the three backtrack triggers are deterministic tests on the scores, and the ban list, not a reflection, is what makes a restored frontier explore differently.

Appendix B.4. Claim routing and the verifier’s outcomes

One model call sees the question, the sixty traversed triples most similar to it, and up to twenty-five candidates. It drafts prose, extracts the entities the draft asserts as its answer, and decomposes the draft into atomic claims over entity *names*, since the model has never seen a graph identifier; the relation slot is left as free text for the reason Appendix B.1 gives.

Each claim is routed through up to three tests and leaves at the first that settles it. **Traversed adjacency** collapses the traversed triples into an undirected set of endpoint pairs; a claim is supported if its pair is in that set, and every traversed triple joining the pair is attached as citable evidence. This is the only route of the three that attaches evidence. **verify_connection** catches claims whose endpoints were both reached but never walked together, querying the full graph for the same relation-agnostic adjacency. Claims that clear neither route, together with those naming an entity the agent never traversed at all, fall to an **entailment check**: one batched model call judging each remaining claim against the same fact window the drafter saw. Symbols decide, and the model is consulted only where the graph is silent.

The router that follows has three outcomes. No unsupported claim routes to **grounded**, and the draft is emitted verbatim with no further model call. An unsupported claim with verification, depth, and time budget remaining routes to **retry**: the verifier appends a repair objective naming the first unsupported claim, anchors the frontier on that claim’s head entity, and control returns to the Explorer. Otherwise the router takes **give_up**, and one rewrite call restates the draft around a *deterministically filtered* entity list, in which an entity survives if it appears in a supported claim or in no unsupported claim. An earlier design let the model regenerate the list itself, and that reintroduced exactly the ungroundedness the layer exists to prevent.

On the 80-question development set used to validate this design, 77 questions asserted no unsupported claim and took the grounded path at zero additional model cost. The remaining

3 took `give_up`, and the retry route did not fire once, because every question that reached the verifier with an unsupported claim had already exhausted its depth budget. On most questions, then, the layer is a pass-through that certifies a draft rather than a mechanism that visibly intervenes, which is the reading Section 6.2 confirms at test scale.

Appendix B.5. Budgets and termination

Routers and the model client enforce every cap; no prompt requests one. A run may take at most 4 expansion depths, a beam of 3 candidate rows per anchor, 5 anchors in the frontier, 3 backtracks, 2 verification iterations, 300 seconds of wall-clock time, and 25 model calls. The depth, wall-clock, and verification caps are checked at two separate sites each, because the `retry` edge re-enters the Explorer without passing back through the router that would otherwise catch it. Every cap is hashed into the run configuration, so a reported result can always be traced to the caps that produced it. The two tool-layer caps of Appendix B.1, 300 candidate relations and 200 neighbours, bound what the agent can see rather than what it can spend, and Section 4.2 shows why they matter for the comparison. Every cycle in Figure B.1 therefore passes through a router that checks or decrements a monotone counter, so no sequence of model outputs, however adversarial, can produce a non-terminating run.

Appendix B.6. Removing a component

Each ablation condition of Section 6 flips one flag and leaves every other line of the loop as it is. **No planner** replaces the planning call with a plan of one objective, the question itself; anchors still come from the annotated topic entities, as they do in the full system. **Embedding-only scoring** sets $\alpha = 1$, which makes no scoring call and ranks candidates by cosine similarity alone. **No backtracking** sets the backtrack budget to zero, so the router ends the search where it would have backtracked. **No verification** leaves the drafting call unchanged, entity list and claim decomposition included, but checks no route, makes no entailment call, and emits the draft verbatim. That condition therefore measures the checking and not the drafting, and its token saving is the cost of the routes rather than of the draft. The budget configuration is hashed into every record, so the condition that produced a row of Table 3 can be recovered from the row.

Appendix C. Protocol detail

Section 4 states what each baseline licenses and how the comparison is controlled. This appendix records how the two non-trivial baselines were built and how groundedness was measured.

Appendix C.1. The static graph baseline

Static GraphRAG [4] fetches a fixed-radius neighbourhood around the linked topic entities in one shot, reranks it by embedding similarity to the question, and answers from the top 30 facts. Two properties of that implementation bound what it licenses, and we state them rather than leave a reader to infer them from its scores. The radius is *one logical hop*: mediators are traversed transparently, so a mediator path costs two graph edges while remaining one hop, and a genuine two-hop chain is outside its reach by construction. And expansion retrieves at most 100 edges per topic entity with no ordering criterion, so on a high-degree entity the retained edges are an arbitrary sample rather than the best or the first; at least one topic entity is truncated on 72.5% of the 800 test questions. The baseline therefore licenses one narrow claim, the one Section 4.2 draws, and not a claim about what a two-hop static retriever would score. That experiment was not run.

Appendix C.2. The agentic baseline

Think-on-Graph [5] performs model-guided beam search over relations and entities, reimplemented on this environment’s tools rather than quoted. Beam width and depth follow the original paper, 3 and 3. At each depth it makes one relation-pruning call per frontier entity, one entity-pruning call per selected relation, and one sufficiency call, so a full depth at width 3 costs up to thirteen calls, and the 25-call cap of Appendix B.5 admits one full depth and part of a second. Two widths beneath the beam are this reimplementation’s own rather than the original’s: it offers the pruning prompt the first 40 relations of an entity and the first 20 neighbours of a relation, against the 300 and 200 AGR sees (Appendix B.5). Reimplementing is a stronger comparison than citing the original figures, because graph, model, budget, and scoring rule are held constant and only the algorithm varies. It also means its numbers are not comparable to the published ones, and we do not present them as such. All baselines were debugged on development data and certified before any test

question ran, against a pre-specified diagnostic targeting the one failure that would be an implementation defect rather than a paradigm limit: retrieval found the gold answer and the system hedged anyway.

Appendix C.3. How groundedness is measured

Groundedness is measured at two levels. The structural tier is deterministic and applies to every asserted entity in all 4,000 records: an entity is grounded if and only if it exists as a node in the environment and connects to a topic entity within four hops. The semantic tier asks the harder question of whether the entity is supported *as the answer*. For a stratified sample of 60 answers per system per dataset, it retrieves the graph’s own shortest connecting paths and has a language model judge whether any path expresses the relationship the question asks about, under the governing instruction that mere connectedness is not support. Because path retrieval takes only the three shortest paths, semantic-tier rates are conservative lower bounds. They are depressed uniformly across systems and read as a between-system comparison rather than as absolute support rates. Section 4.3 reports the judge’s validation against a human annotator, including the threshold it missed.

Appendix D. Secondary measurements

Five measurements are reported here rather than in the body. Each supports a headline the body states; none carries a conclusion of its own.

Appendix D.1. Latency

Section 5.2 reports calls and tokens, both exact from the shared meter. Wall-clock time tells the same story in seconds: a mean of 11.2 and 17.2 per question for AGR against 12.9 and 15.2 for the baseline, and between 1.0 and 2.2 for the three single-call systems. Read those as magnitudes only. Records replayed entirely from a response cache are excluded from those means, but between 3.6% and 15.2% of the two navigators’ calls in the remaining records were still served from the cache, which shortens a cached call to nothing. The figures are therefore lower bounds and do not rank the two navigators against each other at the second.

Appendix D.2. Positioning against published RoG results

The environment of Section 4.1 is built from the per-question subgraphs that the RoG distribution publishes, and RoG [7] is the closest published system in the path-retrieval family, so a reader who notices both will ask how the two compare. The aggregate answer does not favour this work, and we give it before the qualifications. Over the full test splits RoG publishes 85.7 Hits@1 and 70.8 F1 on WebQSP and 62.6 and 56.2 on ComplexWebQuestions, against this work’s 75.5 and 64.2, and 52.2 and 46.9 over the certified 400-question samples of those same splits. **RoG leads on all four figures, and the margin is not sampling noise:** AGR’s 95% intervals on Hits@1 are [71.3, 79.7] on WebQSP and [47.5, 57.0] on ComplexWebQuestions, and RoG’s 85.7 and 62.6 fall outside both.

One difference bounds what that comparison establishes, and it is decisive. **RoG is trained on these benchmarks; AGR is trained on nothing.** RoG fine-tunes LLaMA2-Chat-7B on the training splits of both benchmarks, which hold 2,826 and 27,639 questions in the distribution used here, while AGR’s backbone is frozen and prompted and has seen no question from either benchmark before evaluation. Three ordinary differences follow it: the backbones differ, which is the very factor Section 4.2 holds constant elsewhere; RoG reports over the full splits against our stratified samples of them and publishes no interval, so the gap can be bounded from one side only; and RoG retrieves within each question’s own published subgraph while AGR navigates the union of all of them. What the comparison settles is AGR’s place in the published landscape, a training-free system against a supervised one on the same question sets. What it does not settle is anything this paper measures, since neither the ablation of Section 6 nor the census of Section 7 has a counterpart in RoG’s evaluation.

Appendix D.3. Accuracy by hop depth

Stratifying by the number of hops separating a topic entity from the gold answer separates the systems further. On ComplexWebQuestions, AGR is the only system whose accuracy *rises* with hop count: 0.46 at one hop, 0.55 at two, 0.57 at three or more, over strata of 137, 211, and 49 questions. The last step sits well inside the interval of a 49-question stratum, so the claim is the direction across the three strata, not the size of any one step. Every other system ends below where it started. That phrasing is deliberate and the weaker alternative

Table D.4: Semantic-tier support rate: the share of sampled assertions a judge found supported *as the answer* by the graph’s own connecting paths. $n = 60$ per cell, so the 95% binomial interval has a half-width of roughly 12 to 13 points. These are conservative lower bounds by construction (Section 4.3).

System	WebQSP	CWQ
no-retrieval	63.3%	36.7%
Vector-RAG	61.7%	50.0%
GraphRAG	55.0%	36.7%
Think-on-Graph	61.7%	43.3%
AGR	66.7%	48.3%

is the true one: Think-on-Graph does not decay monotonically but falls from 0.53 to 0.37 and then partly recovers to 0.45. It also beats AGR on that dataset’s one-hop stratum, 0.53 against 0.46, and Section 6.1 identifies the mechanism, since decomposition costs accuracy on questions that do not need decomposing and that stratum is full of them.

The static graph baseline’s decay across these strata is deliberately *not* offered as evidence for the paradigm, because its retrieval radius confounds it, and the evidence that it is a radius effect is in the strata themselves. GraphRAG scores 0.44 on WebQSP’s two-hop stratum, where those questions are largely mediator paths a one-hop expansion does reach, against 0.16 on the ComplexWebQuestions two-hop stratum, where the chains are genuine compositions it cannot reach at all. A retriever failing uniformly on depth would not show that gap. A rising curve is what decomposition would predict, but it is consistent with that mechanism rather than evidence for it, and the WebQSP panel carries no comparable evidence: its three-hop stratum contains four questions, which is why the claim is scoped to one dataset.

Appendix D.4. Semantic-tier support

Section 5.4 reports the structural tier. The semantic tier asks the harder question of whether an asserted entity is supported *as the answer*, and it separates the systems far less. Table D.4 gives every cell rather than a selection, because the ordering differs by dataset and any subset of it misleads.

Every system lands in a 36.7–66.7% band. AGR is highest on WebQSP at 66.7%. **On ComplexWebQuestions it is second**, 5.0 points ahead of the agentic baseline but 1.7

Table D.5: The 64 flagged questions that adjudication cleared, read as consensus rows (one row per consensus answer). The triage family is the pre-pass’s own label for why the systems agreed on a wrong entity. The lower rows count which systems were among those agreeing.

	WebQSP	CWQ
Consensus rows cleared as not a label defect	54	31
filed as echo attractor	36	16
filed as relation confusion	16	3
filed as constraint ignored	0	12
filed as environment gap	2	0
Rows on which the no-retrieval control agrees	15	25
Rows on which both navigators agree	26	14
Echo rows on which the no-retrieval control agrees	13	11
Echo rows on which both navigators agree	20	9

behind Vector-RAG’s 50.0%, so its margin over Think-on-Graph is 5.0 points on both datasets and its lead over *every* system is a WebQSP result only. Neither gap survives the sample: at $n = 60$ per cell a 5-point difference sits well inside the interval, as does Vector-RAG’s 1.7, and no conclusion in this paper rests on either. What the tier establishes is the level rather than the ranking, and the level is the more useful finding, which Section 5.4 states.

Appendix D.5. Consensus rows behind the cleared questions

Table D.5 is the row-level backing for the reading Section 7.2 gives of the 64 questions adjudication cleared. A row is one consensus answer, and a question can carry more than one.

Appendix D.6. The next experiments, and what each would settle

Section 8.4 names four follow-ups in the order they would be run. First, the agentic baseline at AGR’s candidate widths under a sweep of the call cap, with the budget split of Table 2 reported at each setting, which tests whether the affordability reading survives equal access to the graph. Second, replication: every system and every ablation condition run several times, so that a delta of a few hundredths of F1 can be read against the run-to-run variance rather than assumed to exceed it, together with the joint-removal condition

Table D.6: The budget split of Table 2 read within hop strata: n and Hits@1 for the two navigators on the questions the baseline finished and the questions the cap clipped. The four three-hop WebQSP questions and the unreachable strata (12 and 3 questions) are omitted.

Dataset	Hops	Think-on-Graph finished			Think-on-Graph clipped		
		n	Think-on-Graph	AGR	n	Think-on-Graph	AGR
WebQSP	1	175	0.846	0.766	81	0.160	0.728
WebQSP	2	100	0.910	0.890	28	0.321	0.679
CWQ	1	69	0.754	0.609	68	0.309	0.309
CWQ	2	123	0.561	0.610	88	0.102	0.477
CWQ	3+	31	0.613	0.581	18	0.167	0.556

Section 6.3 names. Third, the full test splits, which quadruple the discordant pairs the paired tests see. Fourth, one logging change: persist each accepted claim with the route that accepted it and the triples that matched, which makes the verifier’s precision and recall a number a labelled sample can produce.

Appendix D.7. The budget split within hop strata

Section 5.3 reads the budget split of Table 2 within hop strata, to check that the clipped half is not simply the hard half, and quotes every figure it argues from. Table D.6 gives the full breakdown.

Appendix D.8. The failure census by category

Figure D.2 is the distribution behind Section 7, which names its largest categories in prose and argues from two of the smaller ones.

Appendix D.9. What bounds the two larger nulls

Section 6.2 reports that backtracking and model scoring change nothing detectable. Two implementation facts bound those nulls further, and we record them rather than smooth them over.

The backtracker bans the edges of the most recent explorer pass but restores the highest-scoring snapshot, which is often older, so everything expanded in between stays unbanned (Appendix B.3). Over the main runs, 53 explorer passes re-expanded an anchor-and-relation set the same question had already expanded, and at least 65 of the 262 backtracks restored

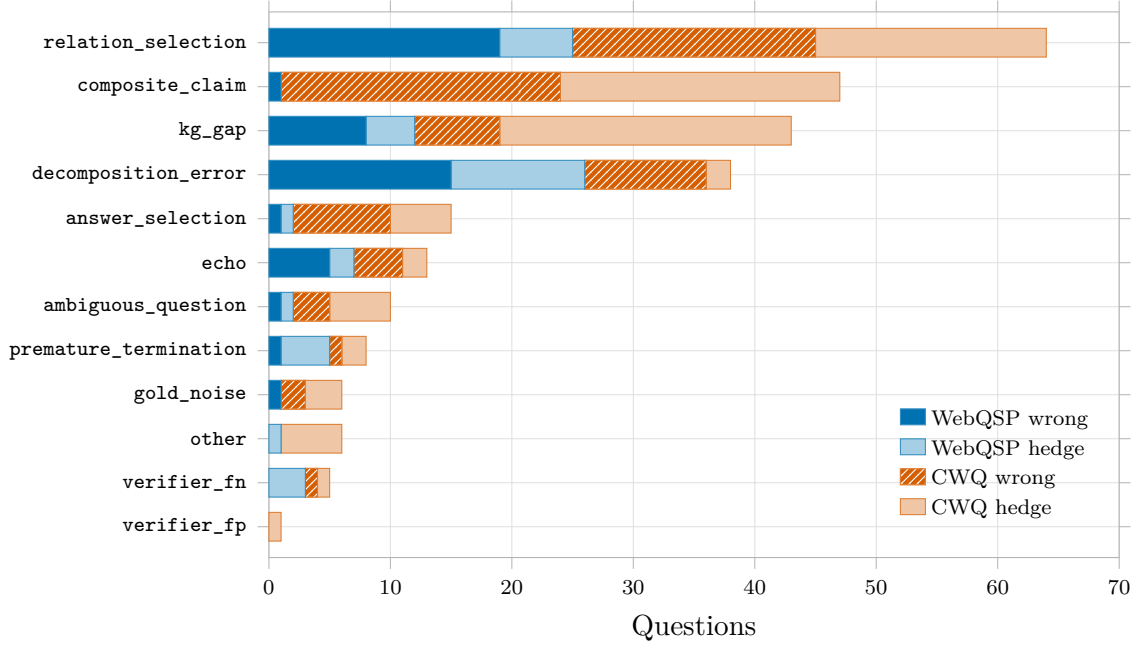


Figure D.2: Hand-read census of the 256 residual failures read, by category, with wrong answers and hedges kept separate. Relation selection and composite claims dominate. The two categories Section 7 discusses are not the largest, but they are the two whose lessons transfer beyond this system.

a depth other than the most recent snapshot’s. The null therefore describes backtracking as implemented, not backtracking as specified.

The development-set sweep that froze α points the same way as the scoring null at a smaller scale: at $\tau = 0.20$ it scored 61, 62, 64, and 59 hits of 80 at $\alpha = 0.3, 0.5, 0.7$, and 1.0, so dropping the model term cost five hits there, before the entity-filter correction that preceded the test runs. One confound goes with it: at $\alpha = 1$ the scorer takes an early return that leaves the backtracking trigger’s signal maximum computed over a smaller candidate set, so that condition’s elevated backtrack count is not attributable to the scoring change alone.

Appendix D.10. What an equal-width re-run would and would not settle

Section 5.3 reports that the agentic baseline searched a narrower candidate pool than AGR, so its figures on the questions it finishes are a lower bound rather than an estimate, and that equalising the widths is the first change we would make in a follow-up. That re-run would not vary one thing, though. Wider candidate sets make each pruning call dearer, so

equal-width pruning would also raise the baseline’s clip rate, moving the boundary between the two halves of Table 2 at the same time as it changes what the search resolves within each. The two effects have to be read apart, which means reporting that split again at each setting rather than an aggregate in which they cancel or compound invisibly.

A related check bounds the aggregate from the other side, because a metric that scores a hit on any match rewards naming more entities. On the WebQSP questions it answers, AGR names 3.37 entities against Think-on-Graph’s 2.56, and its entity precision over those same answered questions is 0.760 against 0.703; on ComplexWebQuestions the figures are 1.54 against 1.40 entities and 0.629 against 0.520 precision. If AGR bought breadth without accuracy, precision would fall as the entity count rose. It does not fall.

Appendix D.11. Which systems joined each cleared consensus

Section 7.2 reports that the systems agreeing on a cleared consensus answer are not independent observers. These are the counts behind that. On ComplexWebQuestions the no-retrieval control, which sees no graph at all, is among the agreeing systems on 25 of the 31 cleared rows and on 11 of the 16 echo rows, so there the convergence is at least as much a property of what the backbone believes as of the graph’s neighbourhood structure. On WebQSP the control joins 15 of 54 rows and 13 of 36 echo rows, and the two navigators agree with each other on 20 of the 36. Of the 13 rows on which all five systems agreed, 9 were cleared as not a defect.

Appendix D.12. The verifier’s unmeasured exposure, in claims

Section 8.1 names wrongful acceptance as the most serious gap in this work and says why the committed record cannot close it. This is its scale in the claim unit. There is no way, from that record, to know how many wrongful acceptances sit inside the 761 questions that ended grounded. On test the verifier decomposed 2,110 claims and accepted 2,008 of them; the graph-wide adjacency probe accounts for 39 of those acceptances, and the rest were made either by traversed adjacency, which is equally relation-blind, or by the entailment check, which is not, and the record does not say which. So the share of accepted claims certified without any test of the asserted relation lies somewhere in $[39, 2,008]$. That interval is two orders of magnitude wide, and we report it as one rather than choose a point inside it. The

same gap means the layer has no measured precision or recall as a classifier of claims, which is the first number a verification component should carry.

The output contract is bounded by the same logging decision. On test, 12 of AGR’s 367 answered WebQSP questions and 9 of its 308 on ComplexWebQuestions were emitted with a count of zero supporting triples, which is the contract’s own measure of how often it delivers nothing.

Appendix D.13. Sensitivity of the tables to the exclusions

Section 7.2 reports that the accuracy tables are not rescored against the adjudicated labels. This is how much that decision could matter. The defects cut both ways: 10 of the 22 WebQSP exclusions and 4 of the 19 on ComplexWebQuestions are questions AGR scored as hits against a label now judged defective. Recomputing every cell of Table 1 with the adjudicated questions removed moves each system by between +0.001 and +0.020 Hits@1 and leaves the ranking on both datasets unchanged. That shift is an order of magnitude below the 5.5% and 4.8% defect rates, because removing a question the system got right subtracts from the numerator as well as the denominator. The exclusions therefore do not threaten any comparison this paper draws. What they establish is that a non-trivial share of both benchmarks is mislabelled, which is a fact about the benchmarks rather than a correction to be applied to them.

CRedit authorship contribution statement

Md. Sakif Khan: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Visualization. **Sadia Sharmin:** Conceptualization, Methodology, Supervision, Project administration, Writing – review & editing.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of competing interest

The authors declare no competing financial interests or personal relationships that could have influenced the work reported here.

Data availability

Code, prompts, per-question run records, and the analysis scripts that produce every figure and table in this paper are available at <https://github.com/sakif-khan/agent-ic-graph-reasoning> under the MIT licence. The question sets are the public WebQSP [9] and ComplexWebQuestions [10] releases, and both they and the Freebase-derived [3] subgraphs arrive through the public Reasoning-on-Graphs distribution [7] (`rmanluo/RoG-webqsp` and `rmanluo/RoG-cwq`), used unmodified. The knowledge environment is rebuilt from that distribution by the scripts in the repository rather than redistributed.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used Claude (Anthropic) to draft and revise the prose of this manuscript from the completed thesis on which it is based, and to write the scripts that check the values quoted in the text against the study’s recorded results. The study design, the experiments, the analysis, and the findings reported here are the authors’ own and predate the preparation of this manuscript. After using this tool the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

References

- [1] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin, T. Liu, A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions, *ACM Transactions on Information Systems* 43 (2) (2025) 1–55.

- [2] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, D. Kiela, Retrieval-augmented generation for knowledge-intensive NLP tasks, in: *Advances in Neural Information Processing Systems*, Vol. 33, 2020, pp. 9459–9474.
- [3] K. Bollacker, C. Evans, P. Paritosh, T. Sturge, J. Taylor, Freebase: A collaboratively created graph database for structuring human knowledge, in: *Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data*, 2008, pp. 1247–1250.
- [4] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, D. Metropolitan, R. O. Ness, J. Larson, From local to global: A graph RAG approach to query-focused summarization, *arXiv preprint arXiv:2404.16130* (2024).
- [5] J. Sun, C. Xu, L. Tang, S. Wang, C. Lin, Y. Gong, L. M. Ni, H.-Y. Shum, J. Guo, Think-on-graph: Deep and responsible reasoning of large language model on knowledge graph, in: *International Conference on Learning Representations (ICLR)*, 2024, *arXiv:2307.07697*.
- [6] L. Chen, P. Tong, Z. Jin, Y. Sun, J. Ye, H. Xiong, Plan-on-graph: Self-correcting adaptive planning of large language model on knowledge graphs, in: *Advances in Neural Information Processing Systems*, Vol. 37, 2024, *arXiv:2410.23875*.
- [7] L. Luo, Y.-F. Li, G. Haffari, S. Pan, Reasoning on graphs: Faithful and interpretable large language model reasoning, in: *International Conference on Learning Representations (ICLR)*, 2024, *arXiv:2310.01061*.
- [8] H. Luo, H. E, Y. Guo, Q. Lin, X. Wu, X. Mu, W. Liu, M. Song, Y. Zhu, L. A. Tuan, KBQA-o1: Agentic knowledge base question answering with monte carlo tree search, in: *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025, *arXiv:2501.18922*.
- [9] W.-t. Yih, M. Richardson, C. Meek, M.-W. Chang, J. Suh, The value of semantic parse labeling for knowledge base question answering, in: *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, 2016, pp. 201–206.

- [10] A. Talmor, J. Berant, The web as a knowledge-base for answering complex questions, in: Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, 2018, pp. 641–651.
- [11] Y. Li, Z. Li, P. Wang, J. Li, X. Sun, H. Cheng, J. X. Yu, A survey of graph meets large language model: Progress and future directions, arXiv preprint arXiv:2311.12399 (2023).
- [12] C. Mavromatis, G. Karypis, GNN-RAG: Graph neural retrieval for efficient large language model reasoning on knowledge graphs, in: Findings of the Association for Computational Linguistics: ACL 2025, 2025, pp. 16682–16699, arXiv:2405.20139.
- [13] M. Li, S. Miao, P. Li, Simple is effective: The roles of graphs and large language models in knowledge-graph-based retrieval-augmented generation, in: International Conference on Learning Representations (ICLR), 2025, arXiv:2410.20724.
- [14] S. Ma, C. Xu, X. Jiang, M. Li, H. Qu, C. Yang, J. Mao, J. Guo, Think-on-graph 2.0: Deep and faithful large language model reasoning with knowledge-guided retrieval augmented generation, in: International Conference on Learning Representations (ICLR), 2025, arXiv:2407.10805.
- [15] L. Luo, Z. Zhao, G. Haffari, Y.-F. Li, C. Gong, S. Pan, Graph-constrained reasoning: Faithful reasoning on knowledge graphs with large language models, in: International Conference on Machine Learning (ICML), 2025, arXiv:2410.13080.
- [16] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, Y. Cao, ReAct: Synergizing reasoning and acting in language models, in: International Conference on Learning Representations (ICLR), 2023.
- [17] S. Dhuliawala, M. Komeili, J. Xu, R. Raileanu, X. Li, A. Celikyilmaz, J. Weston, Chain-of-verification reduces hallucination in large language models, in: Findings of the Association for Computational Linguistics: ACL 2024, 2024, pp. 3563–3578.
- [18] J. Huang, X. Chen, S. Mishra, H. S. Zheng, A. W. Yu, X. Song, D. Zhou, Large

- language models cannot self-correct reasoning yet, in: International Conference on Learning Representations (ICLR), 2024.
- [19] A. Asai, Z. Wu, Y. Wang, A. Sil, H. Hajishirzi, Self-RAG: Learning to retrieve, generate, and critique through self-reflection, in: International Conference on Learning Representations (ICLR), 2024, arXiv:2310.11511.
- [20] S.-Q. Yan, J.-C. Gu, Y. Zhu, Z.-H. Ling, Corrective retrieval augmented generation, arXiv preprint arXiv:2401.15884 (2024).
- [21] S. Min, K. Krishna, X. Lyu, M. Lewis, W.-t. Yih, P. W. Koh, M. Iyyer, L. Zettlemoyer, H. Hajishirzi, FActScore: Fine-grained atomic evaluation of factual precision in long form text generation, in: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, 2023, pp. 12076–12100.
- [22] N. Reimers, I. Gurevych, Sentence-BERT: Sentence embeddings using siamese BERT-networks, in: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing, 2019, pp. 3982–3992.